

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Jorge Arun Zambrano Cedeño, Mg.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: ALVARADO BUSTOS MARIA JANELA

PARALELO: "A"

SEMESTRE: QUINTO

ACTIVIDAD: Actividad #1: Estudio de Caso - Sistema de Gestión de Incidentes (Help Desk)

PERIODO ABRIL - AGOSTO 2026

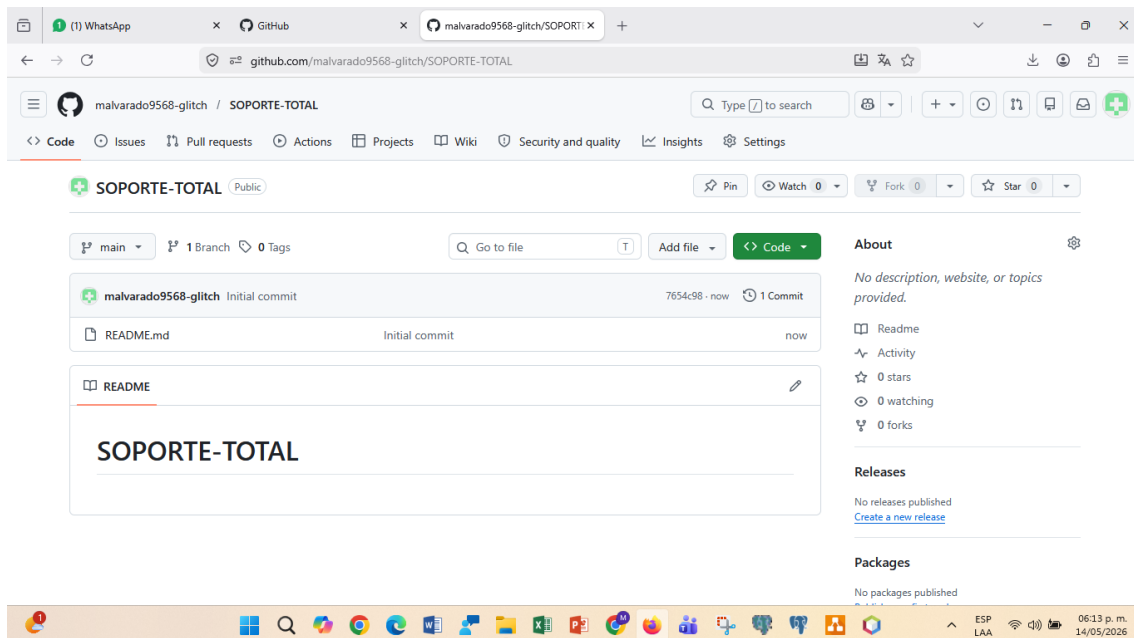


Actividades a desarrollar en la Unidad 1

Actividad # 1

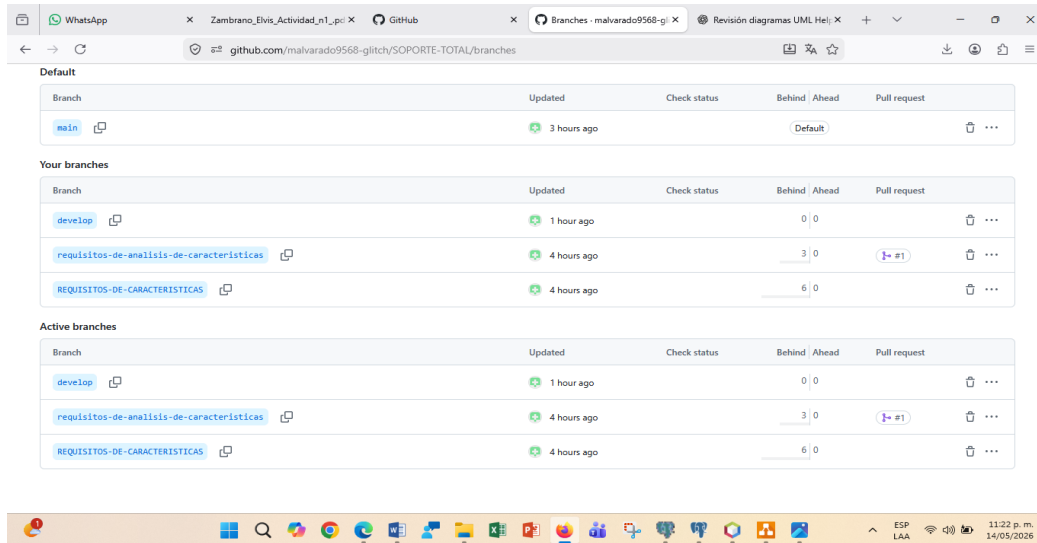
1. Creación del repositorio en GitHub

Se creó el repositorio público SOPORTE-TOTAL en GitHub para gestionar el proyecto Help Desk. El repositorio permite almacenar y versionar todo el avance del análisis del sistema. Dentro de él se creó la carpeta docs para organizar los documentos técnicos. En esa carpeta se subieron los archivos analisis.md y los diagramas UML en PNG. Se utilizó la interfaz web de GitHub mediante la opción "Add files via upload". Esto asegura que el trabajo quede respaldado y accesible desde cualquier lugar. El repositorio servirá como base para aplicar el flujo de trabajoGitflow en las siguientes fases.



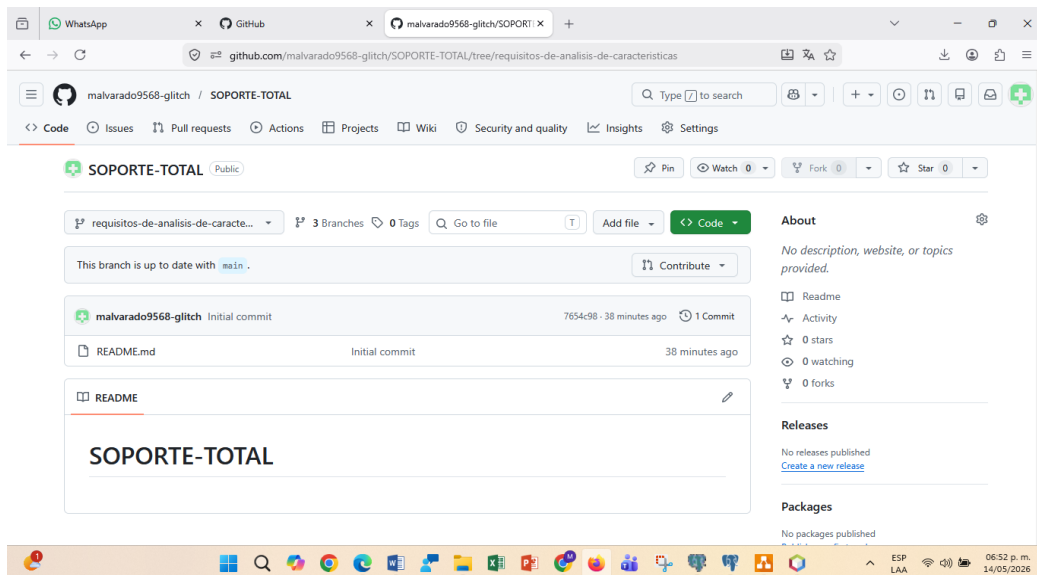
2. Creación de la rama develop con Gitflow

Se creó la rama develop a partir de la rama principal main en el repositorio GitHub. Esta rama sigue el flujo de trabajo Gitflow para separar el desarrollo de la versión estable. En develop se integrarán los avances del análisis y los diagramas del sistema Help Desk. La creación se realizó desde la interfaz de GitHub usando la opción “Create branch”. Esto permite trabajar en paralelo sin afectar el código estable de main. El uso de develop facilita el control de versiones y la colaboración ordenada. Con esto el proyecto queda listo para subir y versionar los documentos del análisis.



Creación de ramas feature con Gitflow

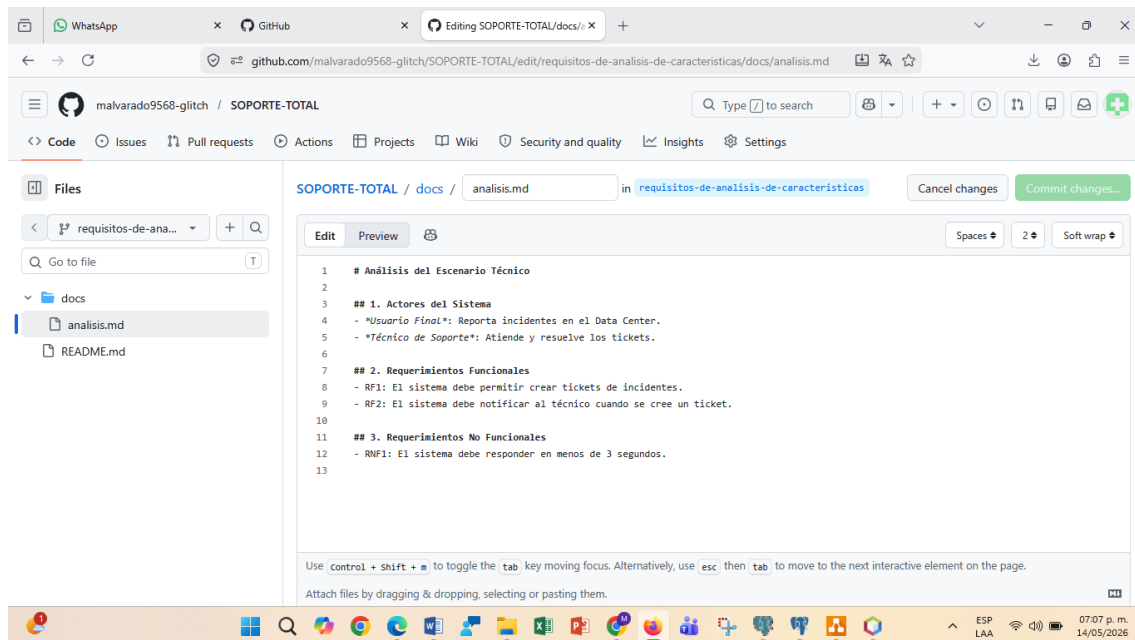
Las ramas feature se crean a partir de develop para desarrollar funcionalidades específicas. Cada rama lleva el nombre de la tarea, por ejemplo feature/analisis-caracteristicas. Esto permite trabajar en una funcionalidad sin afectar a otras ni a la rama principal. Una vez terminada, la rama feature se fusiona de vuelta a develop mediante un Pull Request. Este flujo mantiene el historial limpio y facilita la revisión del código. Se evita trabajar directamente en develop o main para reducir errores. Así se aplica correctamente el modelo Gitflow en el proyecto Help Desk.



Documentación del análisis en la rama feature

En la rama feature/analisis-caracteristicas se documentó el análisis del sistema Help Desk. Se creó el archivo analisis.md con los requisitos funcionales y no funcionales identificados. También se agregaron los diagramas UML de casos de uso, clases y secuencia en formato PNG. Los archivos se organizaron dentro de la carpeta docs para mantener el proyecto ordenado.

Esta estructura permite que el equipo revise y valide el análisis antes de fusionar a develop. Trabajar en una rama feature evita modificar directamente la rama principal de desarrollo. Así se cumple con el flujo Gitflow y se deja un registro claro del trabajo realizado.



Creación del diagrama de casos de uso

Se elaboró el diagrama de casos de uso para representar las interacciones del sistema Help Desk. En él se identificaron los actores principales, como el usuario, el agente y el administrador. Cada caso de uso describe una funcionalidad específica del sistema desde la perspectiva del usuario. El diagrama se realizó para facilitar la comprensión de los requisitos del sistema. Se exportó en formato PNG y se guardó en la carpeta docs del repositorio. Esto permite visualizar de forma clara qué puede hacer cada actor dentro del sistema. El diagrama forma parte de la documentación del análisis y respalda el diseño posterior.

1. Actores

Hay 3 actores identificados:

- Usuario Final: La persona que reporta un problema o consulta el estado de su ticket.
- Técnico: Encargado de atender y resolver los tickets asignados.
- Administrador: Gestiona usuarios, asigna técnicos y genera reportes.

2. Casos de uso por actor

Usuario Final:

- Crear Ticket: Registrar un nuevo problema o solicitud.
- Consultar Ticket: Ver el estado y detalles de sus tickets creados.
- Recibir notificaciones: Recibe actualizaciones del ticket. Está relacionado con "Crear Ticket" mediante <<extend>>, o sea, es opcional y se ejecuta bajo ciertas condiciones.

Técnico:

- Ver tickets asignados: Consultar la lista de tickets que le fueron asignados.
- Actualizar estado: Cambiar el estado del ticket mientras lo atiende. Ej: "En proceso", "Pendiente".
- Resolver ticket: Marcar el ticket como resuelto una vez terminado el trabajo.

Administrador:

- Asignar Técnico a ticket: Asignar un técnico responsable para cada ticket creado.
- Gestionar Usuarios: Crear, editar o eliminar usuarios del sistema.
- Generar reportes: Sacar reportes de tickets resueltos, pendientes, tiempos de respuesta, etc.

3. Relaciones importantes

- El caso de uso Crear Ticket tiene un <<include>> con otro caso de uso que no se ve completo por el corte de la imagen, pero normalmente sería "Validar datos" o "Registrar en BD".
- Recibir notificaciones <<extend>> a "Crear Ticket", significa que no siempre ocurre, solo cuando hay un cambio en el ticket.

DIAGRAMA DE CASO DE USO

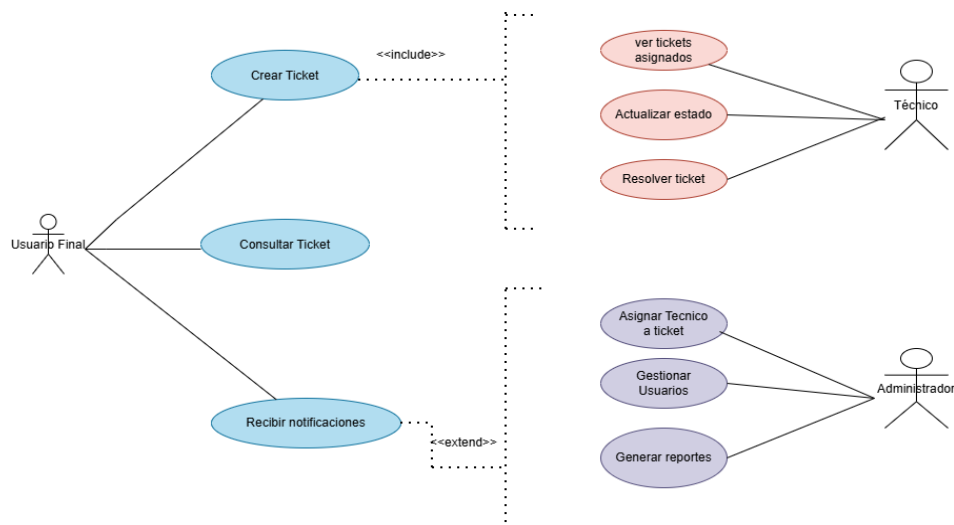


Diagrama de CLASES UML del sistema help desk

El diagrama representa las interacciones entre los usuarios y el sistema Help Desk para la gestión de incidencias. Incluye cuatro actores: Usuario Final, Técnico, Administrador y Sistema de Monitoreo, cada uno con funciones específicas. El Usuario Final puede crear y consultar tickets, además de recibir notificaciones sobre su estado. El Técnico atiende los tickets asignados, actualiza su estado, los resuelve o escala si es necesario. El Administrador se encarga de asignar técnicos, gestionar usuarios y generar reportes de desempeño. El Sistema de Monitoreo detecta



fallas automáticamente y genera alertas que crean tickets en el sistema. Este diseño permite organizar el flujo de trabajo y asegurar una atención ordenada de los requerimientos del sistema.

1. Actores

Hay 4 actores:

- Usuario Final: Quien reporta problemas y consulta tickets.
- Técnico: Atiende y resuelve los tickets asignados.
- Administrador: Gestiona usuarios, asigna técnicos y genera reportes.
- Sistema Monitoreo: Un actor de sistema que genera alertas automáticas sin intervención humana.

2. Casos de uso por actor

Usuario Final - óvalos rosados:

- Crear Ticket: Registrar un problema o solicitud.
- Consultar Ticket: Revisar el estado de sus tickets.
- Recibir notificaciones: Se activa cuando hay cambios en su ticket. Está relacionado con "Resolver ticket" mediante <<extend>>, o sea, es opcional.

Técnico - óvalos verdes:

- Ver tickets asignados: Ver la lista de tickets que le tocan.
- Actualizar estado: Cambiar el estado del ticket mientras trabaja.
- Resolver ticket: Marcar el ticket como resuelto. Este caso <<include>> a "Actualizar estado", porque siempre debe actualizarlo antes de cerrar.
- Escalar ticket: Enviar el ticket a un nivel superior si no puede resolverlo. <<extend>> de "Asignar Técnico".

Administrador - óvalos amarillos:

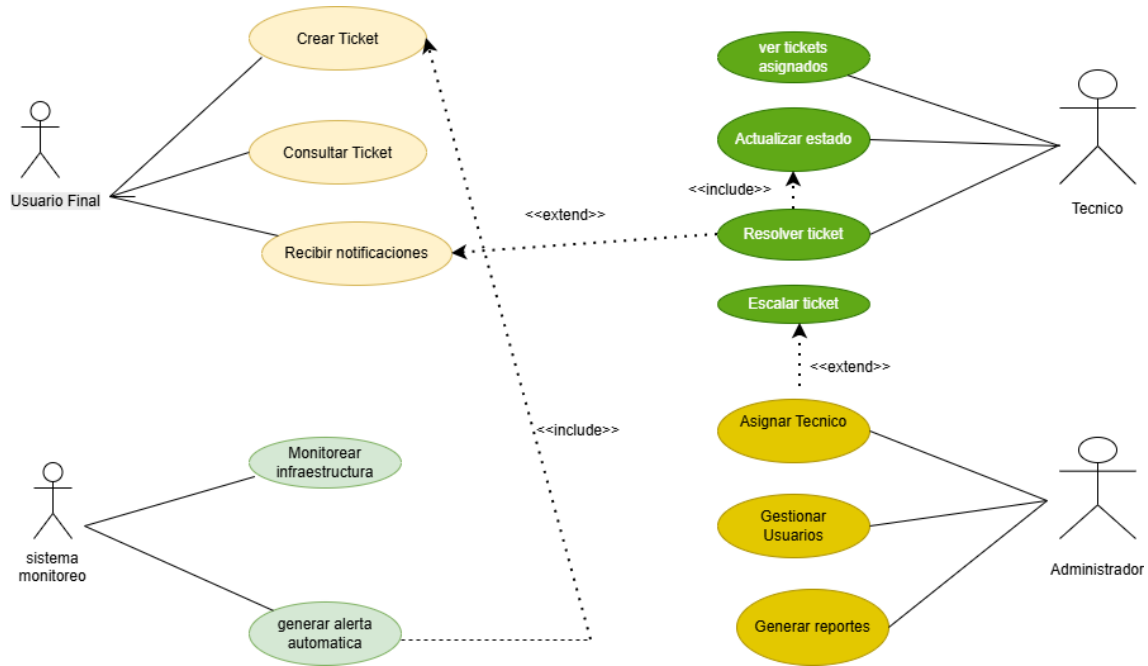
- Asignar Técnico: Asignar un técnico a un ticket nuevo o escalado.
- Gestionar Usuarios: Administrar cuentas de usuarios y técnicos.
- Generar reportes: Sacar reportes de rendimiento y tickets.

Sistema Monitoreo - óvalos azules:

- Monitorear infraestructura: Revisar servidores, redes, etc. constantemente.
- Generar alerta automática: Si detecta una falla, crea un ticket automáticamente. Tiene un <<include>> hacia "Crear Ticket".

3. Relaciones clave

- <<include>>: Relación obligatoria. Ej: "Resolver ticket" siempre incluye "Actualizar estado".
- <<extend>>: Relación opcional que se ejecuta bajo condiciones. Ej: "Recibir notificaciones" se extiende de "Resolver ticket".



ARQUITECTURA GENERAL DEL SISTEMA HELP DESK CON INTEGRACION GITFLOW

El sistema sigue una arquitectura MVC que separa la interfaz, el controlador y la lógica de negocio para mayor orden. El usuario interactúa con la interfaz web/App, que envía las solicitudes al controlador para su procesamiento. La lógica de negocio gestiona la creación, asignación y resolución de tickets, conectándose a la base de datos MySQL o PostgreSQL. Un sistema de notificaciones informa al usuario sobre los cambios en el estado de sus tickets en tiempo real. El código del sistema se gestiona con Gitflow, usando ramas main, develop y feature para organizar el desarrollo. Esto permite trabajar en nuevas funcionalidades sin afectar la versión estable en producción. La integración asegura control de versiones, colaboración ordenada y despliegues seguros del sistema Help Desk.

1. Flujo principal

- Usuario final: Interactúa con el sistema a través de la interfaz web / App. Aquí el usuario crea, consulta y actualiza tickets.
- Controlador (MVC): Recibe las peticiones desde la interfaz. Se encarga de dirigir las acciones y conectar la vista con la lógica del negocio. El código de este controlador viene del repositorio Git, en las ramas main, develop o feature.
- Lógica de negocio (gestor tickets): Contiene las reglas del sistema. Procesa la creación, asignación, actualización y resolución de tickets.

2. Componentes de soporte

- Base de datos MySQL/PostgreSQL: Almacena toda la información de usuarios, tickets, estados y logs. Es el componente de persistencia del sistema.
- Sistema de notificaciones: Se activa desde la lógica de negocio para enviar alertas al usuario cuando cambia el estado de un ticket.

3. Integración con Gitflow

El bloque "Codigo repositorio main/develop/feature" indica que el desarrollo sigue Gitflow. Las ramas feature se usan para nuevas funcionalidades, develop para integración y main para la versión estable en producción.

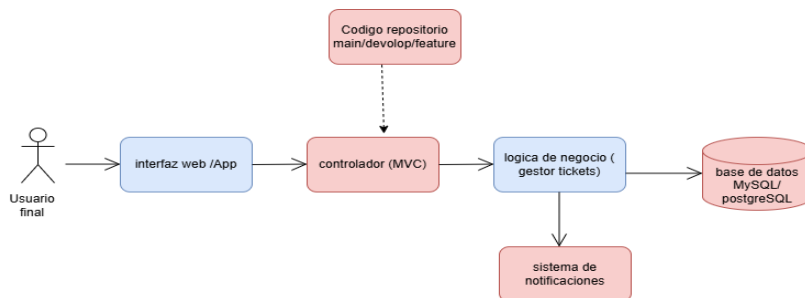


DIAGRAMA DE CLASES

El diagrama modela la estructura del sistema Help Desk aplicando los patrones Singleton, Observer y Factory Method. La clase GestorTickets implementa Singleton para asegurar una única instancia que gestione todos los tickets. Usa el patrón Observer mediante la interfaz IObservador para notificar a los técnicos sobre cambios en los tickets. La clase Tecnico implementa IObservador y recibe las notificaciones con el método actualizar(). TicketFactory aplica Factory Method para crear objetos Ticket sin acoplar el tipo concreto.

La interfaz Ticket define el contrato mostrarDetalles(), implementado por TicketHardware.

Esta estructura promueve bajo acoplamiento, alta cohesión y facilita la extensión del sistema. 1.

Patrón Singleton - GestorTickets

- Clase central con atributo estático instancia y lista observadores.
- Método getInstance() controla el acceso único.
- Métodos agregarObservador(), eliminarObservador() y notificarObservadores() manejan la suscripción y notificación.
- crearNuevoTicket() delega la creación a la fábrica y luego notifica a los observadores.

2. Patrón Observer - IObservador y Tecnico

- IObservador es la interfaz con el método actualizar(mensaje).
- Tecnico implementa esa interfaz y se actualiza cuando cambia el estado de un ticket.
- Esto desacopla al GestorTickets de los objetos que reciben las notificaciones.

3. Patrón Factory Method - TicketFactory y Ticket

- TicketFactory tiene el método crearTicket() que devuelve un objeto que implementa la interfaz Ticket.
- Ticket define mostrarDetalles(). La clase concreta TicketHardware implementa ese método con atributos descripcion, prioridad y estado.
- Permite agregar nuevos tipos de ticket sin modificar el GestorTickets.

4. Relaciones

- GestorTickets conoce a IObservador y usa TicketFactory para crear tickets.
- TicketFactory crea objetos TicketHardware.
- Tecnico implementa IObservador y recibe notificaciones del gestor.

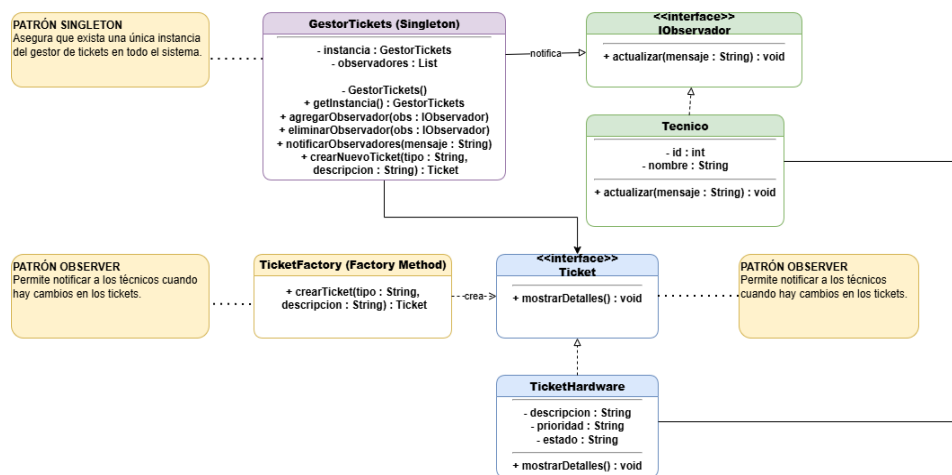


DIAGRAMA DE SECUENCIA

El diagrama muestra el flujo de creación de un ticket de hardware en el sistema Help Desk.

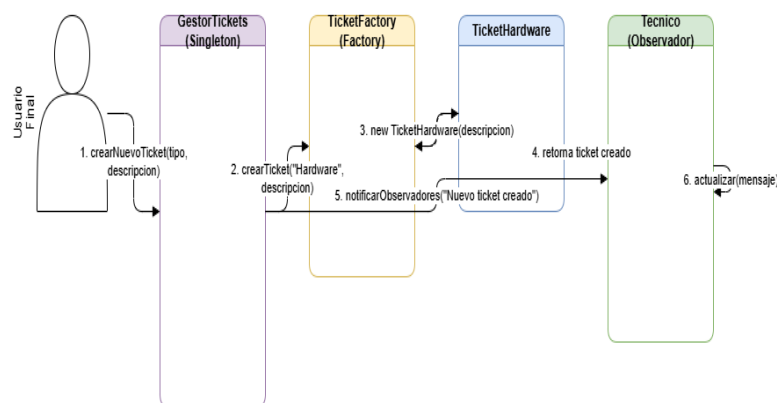
El Usuario Final inicia el proceso llamando a crearNuevoTicket() en el GestorTickets Singleton.

El gestor delega la creación a TicketFactory usando el método crearTicket(). La fábrica instancia un objeto TicketHardware con la descripción proporcionada. Una vez creado, el ticket se retorna al gestor y este notifica a los observadores. El Tecnico, que actúa como observador, recibe la notificación mediante actualizar(mensaje). Este flujo integra los patrones Singleton, Factory Method y Observer en una secuencia clara y desacoplada.

1. Usuario Final → GestorTickets: El usuario invoca crearNuevoTicket(tipo, descripcion). Como GestorTickets es Singleton, garantiza que toda la operación se gestione desde una única instancia.

2. GestorTickets → TicketFactory: El gestor solicita la creación del ticket llamando a crearTicket("Hardware", descripcion), aplicando el patrón Factory Method para no acoplarse a la clase concreta.

3. TicketFactory → TicketHardware: La fábrica crea una nueva instancia de TicketHardware con la descripción recibida.
4. TicketFactory → GestorTickets: Retorna el objeto ticket creado al gestor.
5. GestorTickets → Tecnico (Observador): El gestor notifica a todos los observadores registrados mediante `notificarObservadores("Nuevo ticket creado")`, siguiendo el patrón Observer.
6. Tecnico → `actualizar(mensaje)`: El técnico recibe la notificación y actualiza su estado interno para atender el nuevo ticket.



APLICACIÓN DE PATRONES DE DISEÑO

*Patrón Singleton

Garantiza que una clase tenga una única instancia en todo el sistema y proporciona un punto de acceso global a ella. Se usa cuando un recurso debe ser compartido y controlado centralmente, como la conexión a base de datos. Evita la creación múltiple de objetos que consumen memoria innecesaria. Su implementación controla la instanciación mediante un método estático. En el Help Desk sirve para gestionar la conexión a BD de forma centralizada.

Justificación

Se elige para evitar múltiples conexiones simultáneas a la base de datos que generen sobrecarga. Asegura consistencia en los datos al centralizar el acceso. Reduce el consumo de recursos del servidor. Facilita el mantenimiento al tener un único punto de control. Es útil en entornos con alta concurrencia de tickets.

*Patrón Observer

Define una relación de uno a muchos entre objetos, donde al cambiar el estado de uno, se notifica a los dependientes. Permite desacoplar el sujeto de los observadores. Se usa para implementar sistemas de notificación automáticos. Los observadores se suscriben y reciben actualizaciones sin que el sujeto los conozca directamente. En el Help Desk notifica al usuario cuando su ticket cambia de estado.

Justificación



Permite implementar el sistema de notificaciones sin acoplar la lógica de tickets con la de mensajería. Facilita agregar nuevos tipos de notificación sin modificar el código existente.

Mejora la escalabilidad y mantenibilidad del sistema. Cumple con el principio de responsabilidad única. Mejora la experiencia del usuario con actualizaciones en tiempo real.

***Patrón Factory**

Proporciona una interfaz para crear objetos sin especificar su clase concreta.

Delega la responsabilidad de instanciación a una clase fábrica. Permite crear diferentes tipos de objetos bajo una misma interfaz. Facilita la extensión del sistema con nuevos tipos sin modificar el código cliente. En el Help Desk puede usarse para crear distintos tipos de tickets: soporte, consulta, incidente.

Justificación

Centraliza la creación de objetos y reduce el acoplamiento entre clases. Permite agregar nuevos tipos de ticket sin cambiar el código que los usa. Facilita el mantenimiento y las pruebas unitarias. Respeta el principio abierto/cerrado de SOLID. Hace el sistema más flexible ante cambios de requerimientos.

Importancia de estos patrones utilizados

La aplicación de Singleton, Observer y Factory mejora la organización, escalabilidad y mantenibilidad del sistema Help Desk. Singleton optimiza recursos al centralizar servicios críticos como la conexión a BD. Observer permite un sistema de notificaciones desacoplado y extensible. Factory facilita la creación de objetos complejos y la incorporación de nuevos tipos de ticket. En conjunto, estos patrones siguen principios SOLID y buenas prácticas de ingeniería de software, asegurando un sistema robusto y fácil de evolucionar.

EVIDENCIAS

GestorTickets.java

```
import java.util.ArrayList;
import java.util.List;
```

```
public class GestorTickets {
    // Lista de observadores (Técnicos) a notificar
    private List<IObservador> observadores = new ArrayList<>();

    public void agregarObservador(IObservador observador) {
        observadores.add(observador);
    }

    public void notificarObservadores(String mensaje) {
        for (IObservador obs : observadores) {
            obs.actualizar(mensaje);
        }
    }

    public void crearNuevoTicket(String tipo) {
```



```
System.out.println("\n=== SOLICITUD DE NUEVO TICKET ===");

// Uso del Patrón Factory
Ticket nuevoTicket = TicketFactory.crearTicket(tipo);

if (nuevoTicket != null) {
    nuevoTicket.mostrarDetalles();
    // Uso del Patrón Observer
    notificarObservadores("Se ha creado un nuevo ticket de tipo " + tipo);
} else {
    System.out.println("Error: Tipo de ticket no soportado.");
}
}

// Método Main para probar el sistema
public static void main(String[] args) {
    GestorTickets gestor = new GestorTickets();

    // Creamos técnicos (Observadores)
    Tecnico tecnico1 = new Tecnico("Ana Lopez");
    Tecnico tecnico2 = new Tecnico("Carlos Perez");

    // Suscribimos a los técnicos al gestor
    gestor.agregarObservador(tecnico1);
    gestor.agregarObservador(tecnico2);

    // Simulamos la creación de un ticket de hardware
    gestor.crearNuevoTicket("Hardware");
}
}
```

TicketFactory.java

```
public class TicketFactory {
    public static Ticket crearTicket(String tipo) {
        if (tipo.equalsIgnoreCase("Hardware")) {
            return new TicketHardware();
        }
        return null;
    }
}
```

IObservador.java

```
public interface IObservador {
    void actualizar(String mensaje);
}
```

Tecnico.java

```
public class Tecnico implements IObservador {
    private String nombre;

    public Tecnico(String nombre) {
        this.nombre = nombre;
    }

    @Override
    public void actualizar(String mensaje) {
        System.out.println("Notificando al Técnico [" + nombre + "]: " + mensaje);
    }
}
```



Ticket.java

```
public interface Ticket {  
    void mostrarDetalles();  
}
```

TicketHardware.java

```
public class TicketHardware implements Ticket {  
    @Override  
    public void mostrarDetalles() {  
        System.out.println("Ticket: Problema de Hardware físico registrado.");  
    }  
}
```

The screenshot shows a web browser with multiple tabs. The active tab is 'Upload files - malvarado9568', displaying the GitHub upload interface for the repository 'malvarado9568-glitch/SOPORTE-TOTAL'. A list of files to be uploaded is shown: GestorTickets.java, IObservador.java, Tecnico.java, Ticket.java, TicketFactory.java, and TicketHardware.java. Below this, a 'Commit changes' button is visible.

The second part of the screenshot shows the 'SOPORTE-TOTAL/GestorTicket.java' file in the code editor. The code is as follows:

```
1 import java.util.ArrayList;  
2 import java.util.List;  
3  
4 public class GestorTickets {  
5     // Lista de observadores (Técnicos) a notificar  
6     private List<IObservador> observadores = new ArrayList<>();  
7  
8     public void agregarObservador(IObservador observador) {  
9         observadores.add(observador);  
10    }  
11  
12    public void notificarObservadores(String mensaje) {  
13        for (IObservador obs : observadores) {  
14            obs.actualizar(mensaje);  
15        }  
16    }  
17  
18    public void crearNuevoTicket(String tipo) {  
19        System.out.println("\n=== SOLICITUD DE NUEVO TICKET ===");  
20  
21        // Uso del Patrón Factory  
22        Ticket nuevoTicket = TicketFactory.crearTicket(tipo);  
23  
24        if (nuevoTicket != null) {  
25            nuevoTicket.mostrarDetalles();  
26            // Uso del Patrón Observer  
27            notificarObservadores("Se ha creado un nuevo ticket de tipo " + tipo);  
28        } else {
```





Diagrama sin título - drawio

Diagrama sin título

Archivo Editar Vista Organizar Extras Ayuda Cambios no guardados. Haga clic aquí para guardar

80% 10 pt 10

Diagrama Estilo

Vista

☒ Cuadrícula 10 pt

☒ Vista de la página

Fondo

☐ Color de fondo

☐ Sombra

Opciones

☒ Flechas de conexión

☒ Puntos de conexión

☒ Guías

Tamaño del papel

A3 (297 mm x 420 mm)

Orientación vertical Orientación horizontal

Editar datos...

Quitar estilo predeterminado

Miscelánea

+ Más formas

Casos de Uso

Diagrama

Estilo

Vista

☒ Cuadrícula 10 pt

☒ Vista de la página

Fondo

☐ Color de fondo

☐ Sombra

Opciones

☒ Flechas de conexión

☒ Puntos de conexión

☒ Guías

Tamaño del papel

A3 (297 mm x 420 mm)

Orientación vertical Orientación horizontal

Editar datos...

Quitar estilo predeterminado

arquitectura general de help desk.drawio

arquitectura general de help desk.drawio

Archivo Editar Vista Organizar Extras Ayuda Se han guardado todos los cambios

100% 10 pt 10

Diagrama Estilo

Vista

☒ Cuadrícula 10 pt

☒ Vista de la página

Fondo

☐ Color de fondo

☐ Sombra

Opciones

☒ Flechas de conexión

☒ Puntos de conexión

☒ Guías

Tamaño del papel

A3 (297 mm x 420 mm)

Orientación vertical Orientación horizontal

Editar datos...

Quitar estilo predeterminado

Miscelánea

+ Más formas

Página: 1

Usuario final

Interfaz web (App)

Controlador (MVC)

Logica de negocio (gestor tickets)

base de datos MySQL/postgreSQL

sistema de notificaciones

Código repositorio main/develop/feature

WhatsApp

GitHub

Repositories

github.com/repos

Repositories

Type to search

My contributions

contributed-by:me

0 repositories

No repositories matched your search.

Try a different search query.

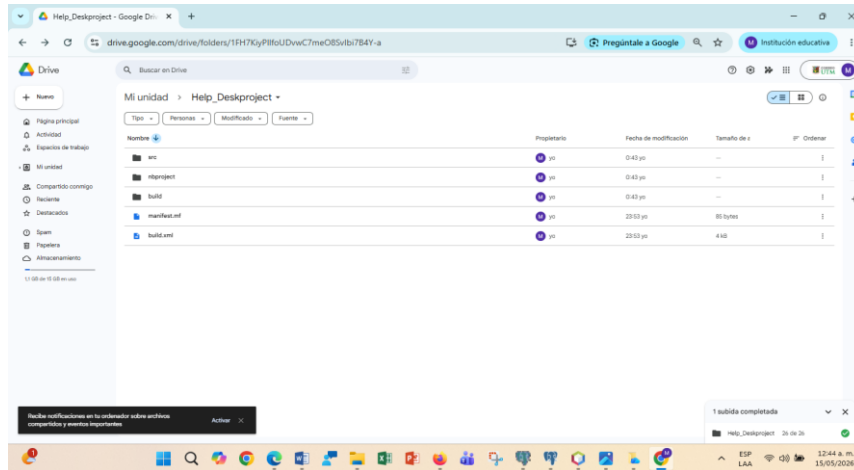
ProTip! When viewing an organization's repositories, you can use the props. filter to filter by custom property.

github.com/repos?q=admin@me

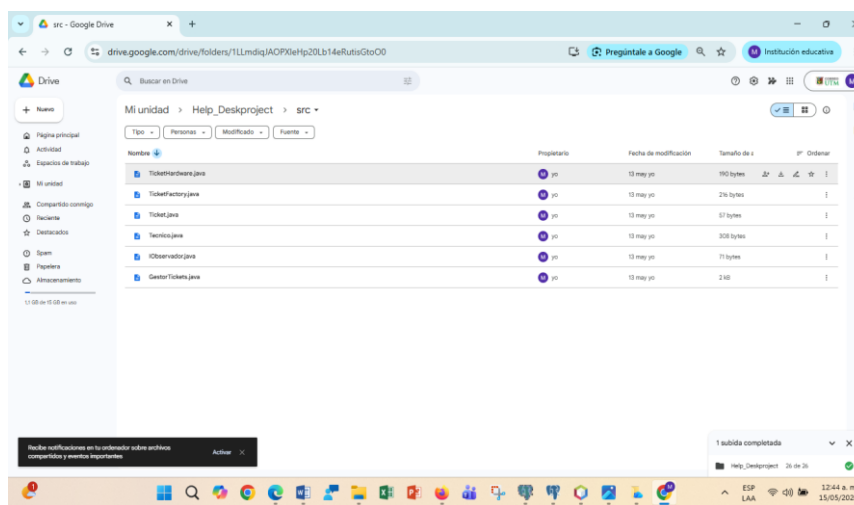
ESP LAA 03:37 p. m. 14/05/2026

ESP LAA 12:24 a. m. 15/05/2026

ESP LAA 09:54 p. m. 14/05/2026



En src esta los codigos



Link de github

<https://github.com/malvarado9568-glitch/SOPORTE-TOTAL>

Link del drive

<https://drive.google.com/drive/folders/1FH7KiyPIlfoUDvwC7meO8SvIbi7B4Y-a?usp=sharing>

BIBLIOGRAFIA

https://www.google.com/search?gs_ssp=eJzj4tLP1TcoqiwoyE1XYDRgdGDwYkvPLMk_oTQIAWC4HFA&q=github&oq=G1&gs_lcrp=EgZiaHJvbWUqEwgBEC4YgwEYxwEYsQMY0QMYgAQyBggAEEUYOTITCAEQLhiDARiHARixAxiRAXiABDINCAIQLhiDARixAxiABDINCAMQABiDARixAxiABDIGCAQQRRg9MgYIBRBFGDwyBggGEEUYPDIGCAcQRRg80gEIMzg5NGowajeoAgiwAgHxBS1OxR28LVBc&sourceid=chrome&ie=UTF-8

https://www.google.com/search?q=draw+io&oq=DRA&gs_lcrp=EgZiaHJvbWUqFAgAEUYOxhdGIMBGLEDGIAEGIoFMhQIABBFgDsYQxiDARixAxiABBiKBTISCAEQABhDGIMBGLEDGIAEGIoFMgwlAhAAGEMYgAQYiqUyEggDEC4YQxiDARixAxiABBiKBTIGCAQQRRg5MgYIBRBFGDwyBggGEEUYPDIGCAcQRRg80gEIMzg4OGowajimoAgawAgHxBV_6gkkNwtm8&sourceid=chrome&ie=UTF-8